

24045922 Manoj Katuwal

 Islington College, Nepal

Document Details

Submission ID

trn:oid::3618:95768832

Submission Date

May 14, 2025, 8:25 AM GMT+5:45

Download Date

May 14, 2025, 8:31 AM GMT+5:45

File Name

24045922 Manoj Katuwal

File Size

18.5 KB

28 Pages

2,732 Words

14,709 Characters

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **47** Not Cited or Quoted 16%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 16%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 47** Not Cited or Quoted 16%
Matches with neither in-text citation nor quotation marks
- 0** Missing Quotations 0%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 0% Internet sources
- 0% Publications
- 16% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
	islingtoncollege on 2025-05-12	3%
2	Submitted works	
	islingtoncollege on 2025-05-13	3%
3	Submitted works	
	islingtoncollege on 2025-05-13	1%
4	Submitted works	
	ESoft Metro Campus, Sri Lanka on 2025-04-27	1%
5	Submitted works	
	islingtoncollege on 2025-05-13	1%
6	Submitted works	
	Study Group Australia on 2012-12-02	<1%
7	Submitted works	
	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-11-04	<1%
8	Submitted works	
	Edith Cowan University on 2024-08-22	<1%
9	Submitted works	
	University of Technology, Sydney on 2006-05-05	<1%
10	Submitted works	
	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-01-19	<1%

11	Submitted works	RMIT University on 2022-10-24	<1%
12	Submitted works	The Robert Gordon University on 2025-04-20	<1%
13	Submitted works	University of Southern Queensland on 2025-05-06	<1%
14	Submitted works	Williams College on 2014-03-15	<1%
15	Submitted works	The Robert Gordon University on 2025-04-20	<1%
16	Submitted works	University of Greenwich on 2016-04-28	<1%
17	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-06-06	<1%
18	Submitted works	RMIT University on 2022-05-20	<1%
19	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-08-16	<1%
20	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-11-02	<1%
21	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-12-31	<1%

Introduction

Introduction of Project

This project was given as the coursework of 1st year students based on real-life scenario of local vendor that sells beauty and skin care products which was named as WeCare. The store sells all types of skin curing disease like which contain salicylic acid, minoxidil acid and many more and beauty products like sunscreen, facewash, hair gel and many more. To manage the products available in a store a simple management system was build in this project. The store has also a given offer Buy 3 get 1 free to increase the more customer. The program also includes the following function that when customer buy 3 products and get 1 free then the stock automatically updates the remaining products as the amount of money only recorded as only included the selling product not the free one.

The application was developed using python and follows the basic data structure for collection and store of the data. The data of the products stored into the txt file which can update, read and write as needed.

Aims and Objective

Main aim of this program is to develop python-based system to manage the products sales and stock for a local vendor. This project helps me to build my strong concept regarding python file handling, exception, function, loops, dictionary, list etc. I learned

to deal with the real-world problem scenario which increase and build my confidence level.

The Objective of this program are as follows:

To develop a simple management system for a local vendor to manage the products.

To arrange the stock of the product which was sold.

To manage the sell product while offer “Buy 3 get 1 free” was given.

To give access to the shop owner to change the price if needed.

Technology used:

 Integrated Development and Learning Environment (IDLE)

IDLE is a default editor that comes with the python. It is a perfect tool for learning the python for beginners. IDLE provides a simple graphical user interface for writing, editing, running python code, syntax highlighting error which helps in debugging the code and fix it. IDLE is so beginner friendly, suitable for small project, easy to write, easy to debug so I have chosen the IDLE (Integrated Development and Learning Environment) for my first ever python project of my Fundamental of Computing project.

Figure 1: IDLE

Data Structure

In this project, data structure are used to store, manage and access product information like product id, name, brand, origin, etc. Primitive data types like string, integer, float are used to hold the value like product name and prices. Non-Primitive data types like dictionary and list are used to store the complex information of the products as dictionary store the key-value pairs and list are used to hold all production as collection.

DATA STRUCTURE

TYPE

DESCRIPTION

EXAMPLE

String

Primitive

Stores text data for names and categories

```
product['name'] = "Aloe Vera Gel"
```

Integer

Primitive

Handles whole numbers for quantities

```
product['quantity'] = 50
```

Float

Primitive

Manages decimal numbers for pricing

```
product['price'] = 12.99
```

Dictionary

Non-Primitive

Key-value pairs for structured product records

```
product = {'name', 'price'}
```

List

Non-Primitive

Ordered collection for multiple product entries

```
products = [{product1}, {product2}]
```

Table 1:Data Structure

Algorithm

An algorithm is a step by step instructions used to solve the problem or to perform a special task. In this WeCare Inventory system, the algorithm handles user interface

satisfaction, manage customer purchases, restock the product and create the invoice of both sale and restock.

Step 1: Start the program.

Step 2: Read product data from products.txt file using read.read_products() method.

Step 3: If product.txt file doesn't exist, it create a new file name products.txt.

Step 4: Store the data into the list name products.

Step 5: Display the main menu with options:-

Display Available Products.

Process Sales Transaction.

Process Restock Transaction.

Exit

Step 6: Prompt user to enter a choice(0-3).

Step 7: If invalid number is selected, it display a message "Invalid choice. Please try again" and return to step 5.

Step 8: If choice is 0 then it return to step 26.

Step 9: If choice is 1(Display Available Products).

Call read.display_Products() method.

Show products details in a well structured table which show the id, product name, brand, origin, stock, cost price and selling price.

Prompt user to like to do another operation or not(y/n).

If y then it return to step 5.

Step 10: If choice is 2 (Process Sales Transaction).

Call read.display_products() method.

Show the products details in a well structured format table.

Prompt for customer name.

Prompt user to select the product id for the purchase of the product.

If the user select the invalid id then it display the message “ Value must be the present

in the product. Please try again” and ask user again to select the id.

If user select 0, then it display the message “No items selected. Transaction cancelled” and ask user again to like to do another operation or not(y/n).

If yes then it return to step 9.

Step 11: Prompt user to the quantity of the product to be purchase.

Step 12: Apply “buy 3 get 1 free policy” and update the quantity in the product list.

Step 13: For the selected product, check if the stock is greater then 0. If not display a message “out of stock” to purchase.

Step 14: Add item to the cart and purchased item list with sub-total and total amount.

Step 15: Generate sales invoice using write.generate_sales_invoice() method along with customer name, purchased items, free items and total amount customer has to pay.

Step 16: Update products.txt file with the modified quantities.

Step 17: Ask user after invoice creation that would you like to perform another operation or not (y/n).

Step 18: If y then return to step 5.

Step 19: If n then to return to step 26.

Step 20: If choice is 3 (Process Restock Transaction).

Display the product list along with their id, name, origin, stock, cost price and selling price of product.

Prompt for supplier name.

Prompt user to select the number of product id which the user wants to restock.

If 0 is selected then return the message “No items selected. Restock cancelled”.

Step 21: If the id of the product is selected then ask the user the quantity to be restock for the product.

Step 22: Ask the user has the cost price has changed or not.

Step 23: If the cost price has changed then you can update the cost price.

Step 24: Generate restock invoice along with the product name, brand, cost, quantity and amount.

Step 25: Update the restock amount in the product.txt file.

Step 26: If choice is 0 (Exit).

Display the message “Thank you for using WeCare Skin Care Product System.

Goodbye!” and exit the program.

Flow Chart

Flow Chart is a visual representation of the algorithm, provide a visual way to understand how the program works and makes easier in designing and explaining

structure of the program. For my project, WeCare Inventory Management system, I have created to show the sequence of my operation in my project.

Figure 2: Flow Chart

Pseudocode

Pseudocode is a simplified half English, half code of writing a program to express the program logic. It helps the developer for planning and explaining the code without the code syntax. For my project, I have created the whole program module pseudocode to explain and express the program logic.

Read module pseudocode

START MODULE

FUNCTION read_products()

INITIALIZE empty product list

TRY

OPEN products.txt file for reading

FOR each line in file

IF line has space remove it

CREATE product dictionary with:

Name = parts[0]

Brand=parts[1]

2

Quantity=converts parts[2] into int

Cost_price= convertd part[3] into float

Origin=parts[4]

APPEND product to product list

ELSE

DISPLAY warning for invaid format

CLOSE File

10

RETURN products list

CATCH FileNotFoundError

DISPLAY error message

RETURN empty list

CATCH other exception

DISPLAY error message

RETURN empty list

END FUNCTION

FUNCTION display_products()

IF product is not found

DISPLAY "No Products Available" message

RETURN

DISPLAY a line separator

DISPLAY the header along with the ID, Product, Brand, Origin, Stock, Cost(Rs) and

Sale(Rs)

DISPLAY a line separator

INITALIZE Index = 1

FOR each product in products

CALCULATE selling_price = product[**cost_price**] *2

DISPLAY index, product name, product brand, product origin, product quantity, product **cost_price** and product selling_price.

INCREMENT index by 1

DISPLAY a line separator

END FUNCTION

END MODULE

Write Module Pseudocode

START write module

FUNCTION write_products(products):

TRY

OPEN product.txt file for writing

FOR each product in products

CREATE formatted line with product name, product brand, product quantity,

product **cost_price** and product_origin.

WRITE line to file

CLOSE file

RETURN true

CATCH any exception

DISPLAY error message

2 FUNCTION generate_sales_invoice(customer_name, items, total_amount)

GENERATE timestamp

CREATE filename with customer_name and timestamp

TRY: OPEN filename.txt file for writing as invoice

WRITE header

WRITE invoice details like date, customer_name, invoice number

WRITE table header with columns: Product, Brand, Price, Qty, Free, Amount

FOR each item in items:

15 WRITE formatted row with item name, item brand, item selling_price, item quantity, item free, item amount

END FOR

WRITE formatted line for footer

WRITE total amount

DISPLAY thank u message

2 CATCH any exception

DISPLAY error message

RETURN none

2 FUNCTION generate_restock_invoice(supplier_name, items, total-amount)

GENERATE timestamp

3 CREATE filename with supplier name and timestamp

TRY:

3

OPEN filename for writing as invoice

WRITE header

WRITE invoice details like Date, Supplier name, and purchased order date

WRITE table header which includes product, brand, cost, qty and amount

FOR each item in items

12

WRITE formatted row with item name, item brand, item cost price, item quantity and amount.

END FOR

WRITE formatted line for footer

WRITE total amount

DISPLAY thank u message

CLOSE file

RETURN filename

CATCH any exception

DISPLAY error message

RETURN none

Operation Module Pseudocode

8

FUNCTION get_valid_integer_input (prompt, min_value=0, max_value=None)

WHILE true

TRY

GET input value from user

IF value < min_value:

DISPLAY error message

ELSE IF max_value is not null AND value > max_value

DISPLAY error message

ELSE

RETURN value

END FUNCTION

CATCH ValueError

DISPLAY error message

FUNCTION process_sales_transaction(products):

CALL display_products(products) method from read module

DISPLAY products

WHILE customer name is empty

GET customer_name from user

IF empty

DISPLAY error message

END IF

END WHILE

END FUNCTION

INITIALIZE empty_sale_item list

INITIALIZE total_amount=0

5 WHILE True

GET product choice (0 to finish)

If choice is 0

4 If saale_itemsA is empty

DISPLAY cancellation message

RETURN false

END IF

BREAK

END IF

GET product from products using choice

1 If product.quantity < =0

DISPLAY out of stock message

CONTINUE

END IF

GET quantity (max product.quantity)

CALCULATE selling_price = product.cost_price *2

1 CALCULATE free_items = quantity /3

CALCULATE total_items = quantity + free_items

1 CALCULATE item_amount = quantity * selling_price

ADD item_amount to total_amount

ADD item to sale_item with details

UPDATE product.quantity by subtracting total_items

DISPLAY confirmation message

END WHILE

GENERATE sales invoice

IF invoice generated successfully

UPDATE products file

DISPLAY success message with total_amount and free_items

RETURN true

END IF

DISPLAY failure message

RETURN false

END FUNCTION

 5

FUNCTION process_restock_transaction(products)

DISPLAY products

WHILE supplier_name is empty

GET supplier_name from user

IF empty

DISPLAY error message

 1

END IF

END WHILE

1 INITIALIZE empty restock_items list

INITIALIZE total_amount = 0

WHILE true

 GET product choice (0 to finish)

 IF choice is 0

 4 IF restock_items is empty

 DISPLAY cancellation message

 RETURN false

 END IF

 BREAK

 END IF

GET product from products using choice

GET quantity

SET new_cost_price = product.cost_price

IF users confirms price change

 GET new_cost_price (must be positive)

END IF

1 CALCULATE item_amount = quantity * new_cost_price

ADD item_amount to total amount.

ADD item to restock_items with details

UPDATE product.quantity by adding quantity

UPDATE product_cost_pricee if changed

DISPLAY confirmation message

END while

GENERATE restock invoice

IF invoice generated successfully

UPDATE products file

DISPLAY success message with total_amount

RETURN true

END IF

DISPLAY failure message

RETURN false

END FUNCTION

END MODULE

Main Module Pseudocode

START main module

FUNCTION display_menu():

DISPLAY formatted menu with options:

Display Products

Process Sales

Process Restock

Exit

END FUNCTION

FUNCTION main()

DISPLAY welcome message

WHILE true:

READ products

IF products is empty

DISPLAY error message

IF user doesn't want to retry

BREAK

END IF

CONTINUE

END IF

DISPLAY menu

TRY:

GET user choice

IF choice is 0

DISPLAY goodbye message

BREAK

ELSE IF choice is 1

DISPLAY products

ELSE IF choice is 2



3

PROCESS sales transaction

ELSE IF choice is 3

PROCESS restock transaction

7

ELSE

DISPLAY invalid choice message

END IF

CATCH ValueError

DISPLAY invalid input message

END try

IF user don't want to continue

DISPLAY good bye message

21

BREAK

END IF

END WHILE

END FUNCTION

END MODULE

Testing

Display product

PROCESS

OUTPUT

Objective

To display the products

Action

Run the program and select 1

Excepted output

Should display product id, product, brand, origin, selling price and cost price

Actual output

Display program id, product, brand, origin, selling price and cost price

Result

The test was successful.

Table 2: test 1

Figure 3:Display Product test

Sell product with invoice test

PROCESS

OUTPUT

Objective

To sell product with invoice file

Action

Run the product and choose 2 , enter the customer name, and select the product which whom you want to buy, enter the quantity of product, select 0 to finish the operation

Excepted output

Product could be sell and generate the sale invoice.

Actual output

Product has been sell and generate sale invoice too.

Result

The test has been successful.

Table 3: sell product test

Figure 4: Sell test

Figure 5: Sell invoice

Buy product with invoice

PROCESS

OUTPUT

Objective

To buy the product.

Action

Run the program, select 3, enter supplier name, enter quantity and enter 0 to finish restocking.

Excepted output

Display buy product in the table and generate restock invoice.

Actual output

Display buy product in the table and generate restock invoice.

Result

The test was successful.

Table 4: buy product test

Figure 6:Product by test

Figure 7:Restock invoice

%

Negative input for quantity (Restock Product Test)

PROCESS

OUTPUT

Objective

To verify the system handles negative value of quantity in restocking product.

Action

Run the program, select option 3 (process restock transaction), enter supplier name, select product id, and input the negative quantity.

Excepted output

Should give error message and ask user again to enter the quantity again.

Actual output

Give error message and ask user again to enter the quantity again.

Result

Successfully handled the program

Table 5: Restock negative input test

Figure 8:Negative value test in restock qunatity

Negative input for quantity (Sales Product Test)

PROCESS

OUTPUT

Objective

To verify the program handles negative value in quantity in selling product.

Action

Run the program, select option 3(process sales transaction), enter customer name, select product id and input negative value in quantity.

Excepted output

Should give error message and ask user to enter the quantity again.

Actual output

Give error message and ask user to enter the quantity again.

Result

Successfully handled by the program

Table 6: negative input for quantity in sales transaction

Figure 9:Negative value test in sales transaction

Quantity more than the stock available

PROCESS

OUTPUT

Objective

To ensure the system prevent customer for purchasing more quantity then is available in stock.

Action

Run the program, choose option 2 (Process Sales Transaction), enter customer name, choose a product id with low stock and attempt to purchase more then the stock.

Expected output

Should give error message and ask user to enter the quantity again.

Actual output

Give error message and ask user to enter the quantity again.

Result

Successfully handled by the program.

Table 7: Quantity more than the stock available

Figure 10:Qunatity test for more than the stock available

User entering product id that doesn't exist

PROCESS

OUTPUT

Objective

Verify that the system detects invalid product id and should display error message.

Action

Run the program, choose option 2 (Process Sales Transaction), enter customer name, chose a product id that doesn't exist.

Expected output

Should give error message and ask user again to enter the quantity.

Actual output

Give error message and ask user again to enter the quantity.

Result

Successfully handled the program.

Table 8:user entering product id that does not exist

Figure 11:User entering product id that doesnt exist

Conclusion

The WeCare Skin product sale system project helps me to build my concept regarding python file handling, function, loops. Through the development of this system I learned how to deal with real word problem scenario, building logic to add "Buy 3 get 1 Free" policy, invoice generation and restocking the product. Overall project helped me to build my problem-solving skill and gain practical knowledge to build a business related software.